A comparison on constrain encoding methods for quantum approximate optimization algorithm

Yiwen Liu*
FinQ Tech Inc.
USA

Qingyue Jiao*
University of Notre Dame
USA

Yidong Zhou
Rensselaer Polytechnic Institute
USA

Zhiding Liang
Rensselaer Polytechnic Institute
USA

Yiyu Shi
University of Notre Dame
USA

Ke Wan
FinQ Tech Inc.
USA

Shangjie Guo
FinQ Tech Inc.
USA

arXiv:2410.04030v1 [quant-ph] 5 Oct 2024

ABSTRACT

The Quantum Approximate Optimization Algorithm (QAOA) represents a significant opportunity for practical quantum computing applications, particularly in the era before error correction is fully realized. This algorithm is especially relevant for addressing constraint satisfaction problems (CSPs), which are critical in various fields such as supply chain management, energy distribution, and financial modeling. In our study, we conduct a numerical comparison of three different strategies for incorporating linear constraints into QAOA: transforming them into an unconstrained format, introducing penalty dephasing, and utilizing the quantum Zeno effect. We assess the efficiency and effectiveness of these methods using the knapsack problem as a case study. Our findings provide insights into the potential applicability of different encoding methods for various use cases.

1 INTRODUCTION

Constraint Satisfaction Problems (CSPs) involve variables that must meet specific constraints. CSPs are crucial in artificial intelligence and operations research due to their ability to standardize various problems, such as the Maximum Cut problem, Knapsack problem, Eight Queens puzzle, Sudoku, and other logic puzzles. They find applications in fields like robotics, production planning, supply chain optimization, employee scheduling, project management, floor plan design, circuit layout, and network configuration.

Solving large CSPs can be computationally intense, especially for NP-complete problems. Researchers are now exploring quantum algorithms as a potential solution. Quantum computing can represent objective functions and potential solutions in superposition [15, 20, 23, 24], allowing simultaneous exploration of all possibilities. This contrasts with classical methods that explore one or a few solutions at a time. Quantum optimization algorithms could greatly enhance the efficiency of solving CSPs.

In quantum optimization algorithms, the most notable algorithms and techniques include the QAOA [16, 17, 22, 27], quantum Annealing [18], and Adiabatic Quantum Computation (AQC) [2]. Also, Grover's algorithm can also be useful to tackle search-related tasks in certain optimization problems.

To effectively leverage quantum optimization algorithms, we need to properly encode constraint information and objective functions. In this paper, we review several alternative methods for encoding optimization problems and designing quantum circuits to solve constrained optimization problems, including quadratization, penalty dephasing, and the quantum Zeno effect.

Review of Quadratization Quadratization is a critical technique in quantum computing, particularly in the context of optimizing high-degree problems by converting them into quadratic forms, which are more amenable to quantum algorithms. This process is essential for various quantum mechanics applications, including quantum annealing and universal adiabatic quantum computing. The work by Dattani in 2019 [7] provides a comprehensive exploration of Hamiltonian transformations that facilitate this conversion, demonstrating the utility of quadratization in discrete optimization problems, crucial for efficient quantum computations.

Further advancements in quadratization methods have been made to improve the accuracy and efficiency of quantum algorithms. Bychkov introduced novel algorithms and software capabilities for quadratization, focusing on non-autonomous dynamical systems of arbitrary dimensions [5]. Their work provides a robust framework for applying quadratization across various disciplines, showcasing its versatility and effectiveness in handling complex optimization tasks.

Another significant contribution to this field is the introduction of the Relaxed-Energy Quadratization method [26], which addresses the inconsistencies of the traditional energy quadratization approach. This method enhances both accuracy and consistency while maintaining the beneficial properties of the original approach, making it a powerful tool for solving gradient flow models and other nonlinear optimization problems in quantum computing.

These research efforts collectively advance the understanding and application of quadratization in quantum computing, enabling more efficient and accurate solutions to complex optimization problems that are central to the field.

Review of penalty dephasing The penalty dephasing is a well-studied approach in quantum circuit design, known for its effectiveness in addressing non-Ising target functions. Rooted in AQC and the adiabatic approximation, this algorithm involves gradually evolving the Hamiltonian from an initial state to a final state that

*These authors contributed to the work equally.

encodes the optimization problem [1][2]. The system, initially prepared in the ground state of the initial Hamiltonian, ideally remains in this state throughout the evolution, reaching the optimal solution. AQC has proven effective for various optimization problems, including Ising spin glasses and satisfiability problems.

Any optimization problem can be reformulated for AQC by constructing a Hamiltonian whose ground state encodes the solution. The quantum system is initially prepared in the ground state of a simple Hamiltonian and evolves adiabatically under a time-dependent Hamiltonian to reach the final Hamiltonian, which encodes the optimal solution.

In related research, Farhi et al. (2014) [11] proposed a quantum algorithm for generating approximate solutions to combinatorial optimization problems. The accuracy of the algorithm improves as the integer p increases. The quantum circuit that implements the algorithm consists of unitary gates whose locality is no more than that of the objective function for which the optimal value is sought. Grand'rive et al. (2019) [8] introduced a relaxed algorithm for solving constrained optimization problems with non-standard Ising target functions. This approach adds penalties for solutions that do not meet the constraints, implemented through separate "cost addition" and "constraint testing and penalty dephasing" blocks, rather than embedding them within the classical Ising Hamiltonian. Building on this work, Lukas et al. (2021) [3] developed a robust implementation that further advances these techniques.

To enable these circuits, quantum networks capable of performing elementary arithmetic operations, including modular exponentiation, were introduced by Vedral et al. (1995) [21]. Additionally, various quantum adders have been developed, such as the quantum ripple-carry adder by Cuccaro et al. (2004) [6], the quantum Fourier transform adder by Draper (2000) [9], and the adder by Ruiz et al. (2014) [19]. Modular exponentiation is a critical operation that dominates both the time and memory complexity in Shor's quantum factorization algorithm.

Review of Quantum Zeno: The quantum Zeno effect, also known as the Turing paradox, is a phenomenon in quantum mechanical systems where continuous measurement or observation prevents a quantum system from evolving, causing it to remain in its initial state. This effect can slow down the time evolution of a particle by frequently measuring it in a specific basis, thereby preventing transitions to states different from its initial state. The quantum Zeno effect arises from the fundamental properties of the Schrödinger equation and is unique to quantum systems, potentially providing novel solutions to a variety of problems.

The quantum Zeno effect has been extensively studied in the literature. Facchi et al. (2009) [10] outline three methods for achieving the Zeno effect: projective measurements, unitary kicks, and strong continuous coupling. They also introduce the concept of Quantum Zeno subspaces through blockwise decomposition. Burgarth et al. (2020) [4] explore the evolution of a finite-dimensional quantum system under frequent kicks, where each kick is a generic quantum operation. They develop a generalization of the Baker-Campbell-Hausdorff formula, revealing an adiabatic evolution. Herman et al. (2022) [13] propose a technique that utilizes quantum Zeno dynamics to solve optimization problems with multiple arbitrary constraints, including inequalities. They demonstrate that the dynamics of quantum optimization can be efficiently confined to the in-constraint subspace through repeated projective measurements. To address the limitations of adiabatic quantum computation, Yu et al. (2022) [25] adopt a Zeno method in quantum simulated annealing, where a series of eigenstate projections are employed. In their approach, the path-dependent Hamiltonian is augmented by a sum of Pauli X terms, whose contributions vanish at the beginning and end of the path.

In this paper, we propose a formal application of the quantum Zeno effect to solve optimization problems, significantly broadening its potential applications. The quantum Zeno effect allows the total Hilbert space to be partitioned into Zeno subspaces, where different components of the density matrix can evolve independently within each sector. By continuously measuring the system, the initial state within one of these subspaces maintains a survival probability of unity. We extend this concept to the QAOA by restricting system evolution to the subspace where feasible solutions reside, rather than embedding all constraints directly into the target function.

The rest of the article is structured as follows: in the Materials and Methods section, we provide a detailed mathematical explanation and implementation of the three methods. In the Results and Analysis section, we evaluate the performance of these methods on knapsack problems with varying problem sizes and parameters.

2 METHODS

2.1 Quantum Approximate Optimization Algorithm

QAOA[11] is a hybrid quantum-classical algorithm widely used to find approximate solutions to combinatorial optimization problems. specifically, QAOA involves the five main steps: problem setup, unitary transformation, ansatz construction, classical optimization, and measurement.

In the first problem setup, we start with a problem that we want to optimize and encode the optimization problem's objective function into a cost Hamiltonian H_C. The goal is to find the ground state of the cost Hamiltonian H_C. Then, unitary transformations are defined by

$$U(H_C, \gamma) = e^{-i\gamma H_C}, \quad U(H_B, \beta) = e^{-i\beta H_B}$$

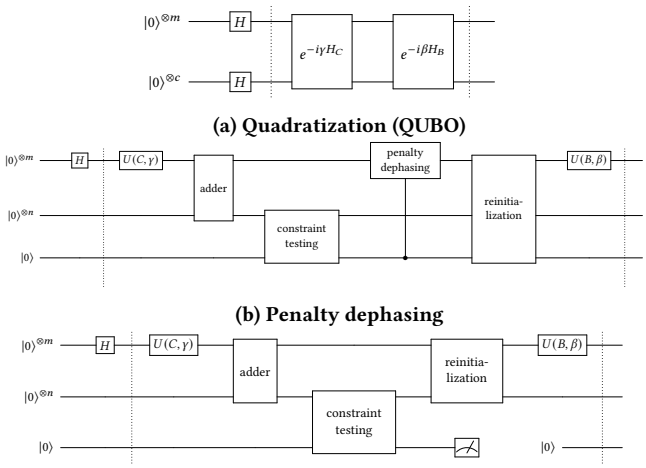
where:

- H_C encodes the optimization problem (objective function).
- H_B is a mixing Hamiltonian.
- Real parameters γ and β embedded in ansatz state need to classically optimized.

In ansatz construction, we need to create an ansatz state ($|\psi(\vec{\gamma}, \vec{\beta})\rangle$) by starting from an initial quantum state $|\psi_0\rangle$ and then applying a series of unitary transformations $U(H_C, \gamma)$ and $U(H_B, \beta)$ on this initial state as below:

$$|\psi(\vec{\gamma}, \vec{\beta})\rangle = U(H_B, \beta_p)U(H_C, \gamma_p) \dots U(H_B, \beta_1)U(H_C, \gamma_1)|\psi_0\rangle$$

where p donotes total number of QAOA layers. Next, we use classical algorithms to optimize parameters $\vec{\gamma} = [\gamma_1, \gamma_2, ...\gamma_p]$ and $\vec{\beta} = [\beta_1, \beta_2, ...\beta_p]$ to minimize the expectation value of the optimization problem Hamiltonian $\langle H_C \rangle = \langle \psi(\vec{\gamma}, \vec{\beta})|H_C|\psi(\vec{\gamma}, \vec{\beta})\rangle$. In this work, we used Constrained Optimization BY Linear Approximation (COBYLA) algorithm designed in scipy library.



(a) Quadratization (QUBO)

(b) Penalty dephasing

(c) Quantum Zeno

Figure 1: One layer circuit structures for constraint encoded variational quantum algorithms: (a) Quadratization QAOA, (b) Penalty dephasing QAOA, and (c) quantum Zeno QAOA. We used Hadamard array for initialization. Vertical dashed lines indicates one QAOA layer.

Finally, the quantum state $|\psi(\vec{\gamma}, \vec{\beta})\rangle$ is measured and outputs a classical bit string. This bit string represents an approximate solution to the original optimization problem.

In the following sections, we will introduce three different approaches for encoding optimization problems with constraints.

2.2 Constraint Encoding Methods for QAOA

In this section we discuss how quantum circuits are designed for each encoding methods, as shown in figure 1. There are seventeen distinct special classes of linear constraints [12]. We choose the most of the special classes which can be reformed into a knapsack type of constrains. The knapsack problem can be summarized as following. Let there be total of m items with index $i = 1, 2, ..., m$ choices for each item $x_i \in \{0, 1\}$, we aim to maximize total value

$$V = \sum_{i=1}^{m} v_i x_i, \quad \text{subject to} \quad \sum_{i=1}^{m} w_i x_i \leq W, \tag{1}$$

where $w_i \in \mathbb{N}$ is the weight of individual item, $W \in \mathbb{N} \geq 2$ is the maximum weight limit, $v_i \in \mathbb{N}$ is the value of individual item.

For all three encoding methods, the first m qubits are dedicated to encoding the decision binary variable $x_i \; \forall i \in [1, m]$. $x_i = 1$ indicates the selection of ith knapsack, and $x_i = 0$ otherwise. Consequently, each vector $x = [x_1, ..., x_m]$ represents a distinct selection scenario.

To calculate the number of ancillary qubits required to represents the constrains term, we denote $c = \lceil \log_2(W) \rceil + 1$, where $\lceil\rceil$ denote for ceil function. In the case that $\exists k \in \mathbb{N}, \; W = 2^k$, we can use the last ancilla qubit to indicate if the constrain is satisfied. And in the case where W is not powers of 2 but an arbitary integer, we can use a trick to add a constant such that it can be treated as a powers of 2:

$$\sum_{i=1}^{m} w_i x_i < W \Leftrightarrow \sum_{i=1}^{m} w_i x_i + s < W + s = 2^c \tag{2}$$

We can turn value 2^c into binary representation$\sum_k 2^k c_k$, where k is the number of binary digits needed. Upon introducing a slack variable, the problem is equivalent to minimizing the following cost function:

$$f = -\sum_{i=1}^{m} v_i x_i + P(\sum_{i=1}^{m} w_i x_i + \sum_{k=0}^{c} 2^k c_k - W)^2 \tag{3}$$

where $P \in \mathbb{R}$ is the Lagrange multiplier determining the importance of our constrains.

2.2.1 Quadratization. The first approach we tested is to translate CSP into QUBO problem by relaxing the linear constrains to quadratic terms into objective functions. For the knapsack problem, we form a new unconstrained objective function as below: Next we can map QUBO objective function 3 into an Ising Hamiltonian with form:

$$H'_C \quad = \quad \sum_{a} Q_{ij} \frac{(\mathbb{I} + Z)_i}{2} \frac{(\mathbb{I} + Z)}{2} + \sum_{i=1}^{m} B_i \frac{(\mathbb{I} + Z)_i}{2} \tag{4}$$

by performing the mapping: $x_i \rightarrow \frac{1}{2}(\mathbb{I} + Z)_i, \quad i = 1, ..., n$, where Z stands for Pauli Z operator. And matrix Q and vector B is calculated by:

$$Q_{ij} = \begin{cases} 2w_i w_j P & \text{if } 1 \leq i < j \leq m \\ 2^{i+j-2m+1} P & \text{if } m+1 \leq i < j \leq m+c \\ w_i 2^{j-m+1} P & \text{if } 1 \leq i \leq m \text{ and } m+1 \leq j \leq m+c \end{cases} \tag{5}$$

and

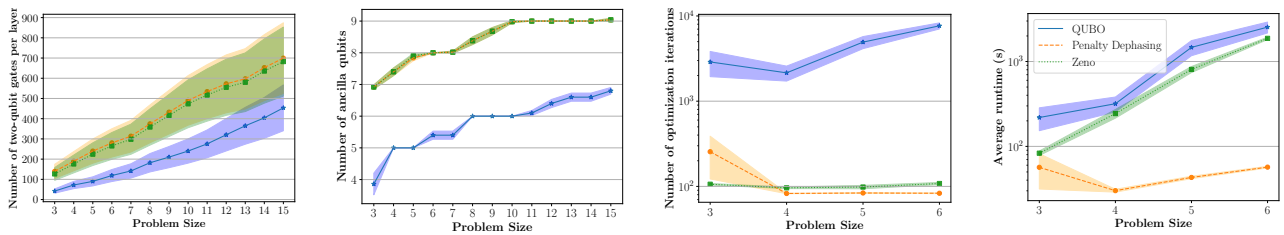
$$B_i = \begin{cases} -v_i + P(w_i^2 - 2W w_i) & \text{if } 1 \leq i \leq m \\ P[4^{(i-m)} - W 2^{(i-m+1)}] & \text{if } m+1 \leq i \leq m+c \end{cases} \tag{6}$$

To simplify this we have used of the fact that Q_{ij} is diagonal symmetric in the first term, and Z_i commutes with Z_j. And we dropped the last term is a constant term, with no effect on the eigenstates. We are only interested in constructing a quantum circuit corresponding to matrices Q and B for implementing $\exp(-i\gamma H'_C)$.

To construct H'_B for implementing the operator $\exp(-i\beta H'_B)$, we use the Pauli X operator array $H'_B = \sigma_x^{\otimes m+c}$, where $m + c$ is the number of qubits needed for encoding the full objective function. The overall quantum circuit for quadratization QAOA is shown in figure 1a.

2.2.2 Penalty dephasing. The second approach to encoding the constraint in the Knapsack problem utilizes penalty dephasing[8]. In penalty dephasing, the full objective function is divided into two distinct components: the return part and the constraint penalty part. The return part consists of the original objective function: $f_{\text{return}} = -\sum_{i=1}^{m} x_i v_i$ and a penalty phase will be added to a solution $\vec{x}$ when $\sum_{i=1}^{m} w_i x_i - W > 0$. The penalty dephasing circuit is comprised of six main components, as shown in Figure 1b.

The first block encodes the return part. The value of each individual knapsack v_i is normalized and reflected by the phase of each



(a) No. two-qubit gates per layer **(b) No. ancilla qubits** **(c) No. optimization iterations** **(d) Average simulation runtime**

Figure 2: Quantum resources required for different problem sizes. (a) Number of CNOT gate for each QAOA layer after decomposition; (b) Number of ancilla qubits needed in addition to data quantum register; (c) Number of optimization iterations for classical optimizer to converge; (d) Total run time for classical simulation of the QAOA algorithm. All plot are generated averaging over 50 random problems with a fixed number of QAOA layers $p = 5$. The shaded area denotes the distribution of 25%-75% of samples. Y-axes in figures (c) and (d) are drawn on a log scale for better illustration.

corresponding qubit. With phase gate

$$\mathrm{P}_i(\lambda) = \begin{bmatrix} 1 & 0 \\ 0 & e^{i\lambda} \end{bmatrix} \tag{7}$$

for the i-th qubits, then the return operator $U(H_C, \gamma)$ is can be represented as

$$\otimes_{i=1}^{n} \mathrm{P}_i\left(-\gamma * v_i\right). \tag{8}$$

The second block constructs a weight adder to verify the knapsack constraint. Once a certain knapsack selection scenario is reached, the cost, the total weight of selected knapsacks in this case, is calculated by the weight adder. Ancillary qubits are allocated for the addition operation. The number of ancillary qubits $n = \lceil \log_2(\sum_{i=1}^{m} w_i) \rceil + 1$ should be sufficient to represent binary decomposition of summation of all item weights.

The adder unitary is defined such that:

$$\mathrm{ADD}\,|x_1, ..., x_n\rangle \otimes |0\rangle := |x_1, ..., x_n\rangle \otimes \left|\sum_{i=1}^{m} w_i x_i\right\rangle, \tag{9}$$

which can be simplified as

$$\mathrm{ADD}\,|\vec{x}\rangle = |\vec{x}\rangle \otimes |\mathrm{Weight}(\vec{x})\rangle. \tag{10}$$

where $|\mathrm{Weight}(\vec{x})\rangle$ is the binary decomposition of the total weight by m ancillary qubits. For implementation, We use quantum Fourier transform (QFT) adder to add weight for all senario. [19]

The third block of the circuit verifies the satisfaction of the constraints and records the results in a flag qubit, which is defined as:

$$\mathrm{TEST}\,|\mathrm{Weight}(\vec{x})\rangle \otimes |0\rangle = |\mathrm{Weight}(\vec{x})\rangle \otimes |\text{if Weight}(\vec{x}) > W\rangle. \tag{11}$$

Such that flag qubit indicates a constraint violation after the constrain testing protocol.

The fourth block of the circuit imposes a penalty on all infeasible solutions by applying controlled phase gates to all qubits in the first block. The control qubit is the flag qubit. The penalty dephasing effectively creates the inference and decreases the likelihood of infeasible solutions. The magnitude of the penalty phase is determined by the extent to which the weight exceeds the limit:

$$\mathrm{PENAL}\,|\psi\rangle \otimes |a\rangle = e^{-ia\alpha}\,|\psi\rangle \otimes |a\rangle \tag{12}$$

where $a \in \{0, 1\}$ is valur of flag qubit and α is a positive, tunable parameter representing the strength of penalty.

The fifth block reinitializes all ancilla registers to prepare for the next calculation, i.e.

$$\mathrm{REINIT} = (I \otimes \mathrm{TEST}^{\dagger})(\mathrm{ADD}^{\dagger} \otimes I). \tag{13}$$

The final mixer operator $U(H_B, \beta) = \exp(-iH_B\beta)$ is implemented with single-qubit X gates as the previous approach, but just on data register, which is expressed as $H_B = \sigma_x^{\otimes m}$.

2.2.3 Quantum Zeno. This approach is inspired by the fact that frequent measurements on a quantum system can project the system onto a reduced subspace. By frequently measuring the flag qubit, which indicates whether the solution (in superposition) is feasible or not, we can potentially confine the quantum evolution within a quantum Zeno subspace. This behavior, in turn, may effectively enforce the constraint, ensuring that it remains satisfied as the quantum state evolves [14].

To illustrate how quantum Zeno effect works, let us examine a quantum system in a finite-dimensional Hilbert space $\mathbb{H}$ governed by the unitary operator $U(t) = e^{-iHt}$, where H is the time-independent Hamiltonian of the system. Assume a weak measurement is represented by a set of orthogonal projection operators P, which projects the time evolution on to the subspace where all constraints are satisfied. The measurement determines whether the system is in the subspace $P\mathbb{H}$. If the initial density matrix of the system is set as $\rho_0 = |\psi_0\rangle\langle\psi_0|$, where $|\psi_0\rangle$ belongs to the subspace $P\mathbb{H}$, the state of the system at time t is given by:

$$\rho_t = U(t)\rho_0 U^{\dagger}(t)$$

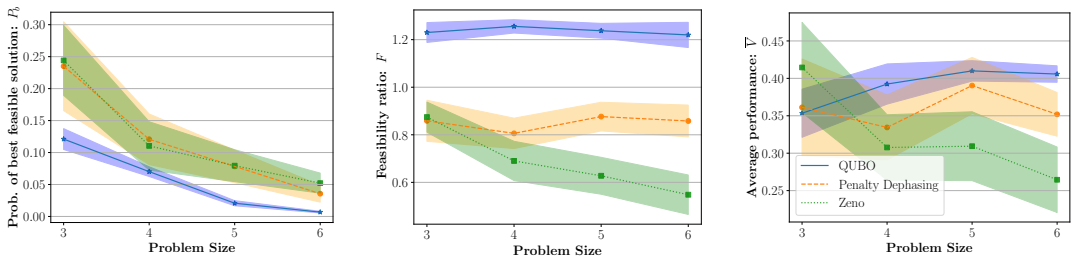
Assume that P and H does not commute, then the state no longer belongs to $P\mathbb{H}$. If we measure with projector P then

$$\rho_t \longrightarrow P\rho_t P = PU(t)\rho_0 U^{\dagger}(t)P = V(t)\rho_0 V^{\dagger}(t)$$

where $V(t) \equiv PU(t)P$. Next we examine the case where we measure N times with P during period of time t:

$$\rho_t^{(N)} = V_N(t)\rho_0 V_N^{\dagger}(t) \qquad V_N(t) \equiv [PU(t/N)P]^N$$

in the limit $N \longrightarrow \infty$, we can Taylor expand $U(t)$ and drop the second and higher order terms:



(a) Prob. of best feasible solution **(b) Feasibility ratio** **(c) Average performance**

Figure 3: Numerical performance for problem sizes from 3 to 6 with fixed number of layers $p = 5$, measure by different metrics: (a) statistical probability of best feasible solution P_B; (b) Feasibility ratio F; and (c) Average performance V. The shaded area denotes the distribution of 25%-75% of samples.

$$\begin{aligned}\lim_{N\to\infty} V_{\boldsymbol{N}}(t) &= \lim_{N\to\infty} [Pe^{-iH(t/N)}P]^N \\ &= \lim_{N\to\infty} \left[P\left(1 - iH\frac{t}{N} + O\left(\frac{1}{N^2}\right)\right)P\right]^N \\ &= e^{-iPHPt}\end{aligned}$$

From the calculation, we conclude that the Zeno Hamiltonian projected on $P\mathbb{H}$ subspace: $H_Z \equiv PHP$, where the time evolution is constrained to.

When constructing the quantum circuit for our knapsack optimization problem, we leverage the subroutines in penalty dephasing including return, adder, constraint testing, and mixer. For reinitilazation we only need to revert adder such that: REINIT = ADD†. And constrain enforcing with Zeno, we take one measurement at the flag qubit and reprepare it to $|0\rangle$. Note that we do not post-select based on these extra measurement results, since this post-selection dramatically increase the number of shots for quantum Zeno and we would like to compare each encoding methods fairly.

3 RESULTS AND ANALYSIS

In this section, we present the results of applying three constraint encoding methods to the Knapsack problem. All experiments were conducted using classical simulation run on commercial laptops.

To evaluate the results, we focus on two key aspects: quantum resource utilization and numerical performance. We analyze the number of ancilla qubits, the number of two-qubit gates per layer, the number of optimization iterations, and the average runtime in seconds for a single problem instance.Three distinct metrics are designed to emphasize different aspects of the results. The first metric, P_B, represents the probability of obtaining the best feasible solutions, which is a critical indicator when only the optimal solution is of interest. The second metric, feasibility ratio, is defined as

$$F = \frac{\sum P(\text{feasible solutions})}{\text{Number of feasible solution}} \times \text{Number of all solutions}$$

The feasibility ratio is particularly useful in scenarios where ensuring feasibility is the primary objective. This metric is designed to account for varying problem sizes and the number of solutions in randomly generated knapsack problems. The final metric is the average performance

$$\overline{V} = \frac{\sum_{\text{feasible solutions}} \text{P(solution)} \times \text{Solution value}}{\text{Best solution value}}$$

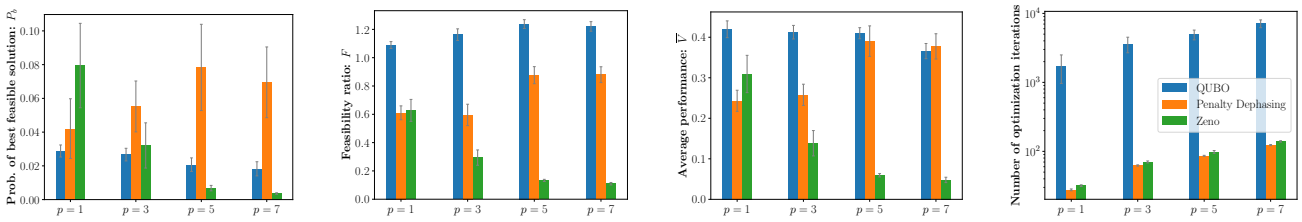
This metric provides a more balanced evaluation, capturing good average performance. All evaluations are averaged over 50 randomly generated problem instances, with the shaded error regions representing 25% of the standard deviation.

3.1 Sensitivity w.r.t problem size

We fix the number of layers p at 5. The penalty Lagrange multiplier P for quadratization is set to be 10 and the penalty weight α for penalty dephasing is set to be 10000. Both quantum resources and performance are evaluated as the size of the knapsack problem varies. The problem size is defined by the number of items in the knapsack.

Figure 2 presents the results of quantum resource usage. As expected, the number of ancilla qubits and two-qubit gates increases with the problem size. Quadratization (QUBO) requires fewer ancilla qubits and two-qubit gates compared to the other two methods, as the constraints are embedded into the objective function through penalty terms. In contrast, penalty dephasing and Zeno often involve additional quantum operations to manage constraints during computation, such as dynamically adjusting penalties or performing mid-circuit measurements, respectively.

Although QUBO uses fewer ancilla qubits and two-qubit gates, the results indicate that it requires a greater number of optimization iterations and a longer runtime. A potential explanation for this is that, while QUBO simplifies the quantum circuit by incorporating penalty terms directly into the cost function and reduces the number of qubits, it also complicates the optimization landscape. The optimization process must navigate a more rugged and complex cost function, which may contain numerous local minima due to the penalty terms. Another notable observation is that the average runtime for Zeno is significantly longer than for penalty dephasing, even though Zeno requires fewer optimization iterations. This might be attributed to non-linear mid-circuit measurements which make the states in Zeno circuits mixed states. Simulating



(a) Prob. of best solution **(b) Feasibility ratio** **(c) Average performance** **(d) No. optimization iterations**

Figure 4: Numerical performance and optimization complexity with varying p and fixed problem size of 5, measure by different metrics: (a) statistical probability of best feasible solution P_B; (b) Feasibility ratio F; (c) Average performance V; and (d) Number of optimization iterations for classical optimizer to converge (y-axis in log scale).

mixed states is more time-consuming than simulating pure states, as mixed states are represented by density matrices that occupy a larger computational space, rather than state vectors as in the case of pure states. Consequently, the simulation for Zeno circuits takes longer.

Figure 3 presents the performance of the three methods. The three metrics illustrate the trade-off between feasible solutions and optimal solutions. Penalty dephasing and Zeno exhibit lower feasibility ratios F, indicating that they are less constrained and more likely to find solutions. In contrast, QUBO imposes stronger constraints on the solution space, which likely creates a more rugged and complex optimization landscape, making it more challenging to identify the best solutions. However, the average performance metric suggests that the more constrained QUBO method yields slightly better average performance. This is a reasonable outcome, as most solutions identified by QUBO are feasible, even if they do not have the highest values. These metrics highlight potential differences in the encoding methods, providing insights into their distinct usage scenarios.

3.2 Sensitivity w.r.t number of layers

We investigate the change of quantum resource usage and performance as the number of layers p changes across the three methods, with the problem size fixed at 5. Since the number of ancilla qubits and two-qubit gates are analyzed on a per-layer basis, they will not be included in the following discussion.

Figure 4 illustrates the changes in performance as the number of layers increases. For the probability of finding the best feasible solution (P_B), penalty dephasing follows the expectation that as the number of layers increases, the circuit becomes more robust, leading to an increase in P_B. However, QUBO and Zeno exhibit the opposite behavior. A possible explanation for this is that as p increases, the optimization landscape becomes more rugged and complex, increasing the likelihood of encountering barren plateaus under the current set of parameters. The suboptimal performance of Zeno may be due to the absence of post-selection. Post-selection was deliberately excluded from the analysis, as it significantly impacts runtime as well as accuracy and we want to control a similar QAOA procedure to compare the encoding methods.

The feasibility ratio for QUBO and penalty dephasing indicates that these methods impose stronger constraints on feasible solutions as p increases. Zeno, on the other hand, continues to show suboptimal results with increasing p. The average performance of QUBO remains relatively unchanged, likely because the feasibility ratio and probability of finding the best solution shift in opposite directions as p increases. Penalty dephasing, however, demonstrates the best average performance at 5 layers, which may represent a balance point between finding feasible solutions and identifying the best solutions.

4 CONCLUSION

In conclusion, our comparative analysis of constraint encoding methods for the QAOA provides valuable insights into their effectiveness and practical implications. The sensitivity analysis with respect to problem size and the number of layers revealed key differences in quantum resource utilization and performance metrics.

The QUBO approach demonstrated lower quantum resource requirements, such as fewer ancilla qubits and two-qubit gates, but required more optimization iterations and had longer runtimes. This is likely due to the increased complexity of the optimization landscape from embedding constraints directly into the objective function, making the search for optimal solutions more challenging. Despite these difficulties, QUBO showed slightly better average performance in finding feasible solutions, though it struggled with scalability as problem size increased.

Penalty dephasing improved the probability of finding the best feasible solutions as the number of layers increased but required more quantum resources and didn't always outperform QUBO in feasibility ratio, highlighting a trade-off between resource usage and solution quality. The Quantum Zeno effect, while theoretically promising, encountered practical challenges due to the complexity of frequent mid-circuit measurements, leading to longer runtimes and suboptimal performance, likely from the increased computational burden of simulating mixed states.

Overall, our results suggest that the choice of constraint encoding method should be carefully tailored to the specific characteristics of the problem at hand, considering factors such as problem size, desired accuracy, and available quantum resources. This analysis provides a foundation for future research and practical applications, where these findings can guide the development of more efficient quantum algorithms for complex constraint satisfaction problems.

REFERENCES

[1] Dorit Aharonov, Wim Van Dam, Julia Kempe, Zeph Landau, Seth Lloyd, and Oded Regev. 2008. Adiabatic quantum computation is equivalent to standard quantum computation. *SIAM review* 50, 4 (2008), 755–787.

[2] Tameem Albash and Daniel A Lidar. 2018. Adiabatic quantum computation. *Reviews of Modern Physics* 90, 1 (2018), 015002.

[3] Lukas Botsch. 2021. https://github.com/lbotsch/ibm-quantum-challenge-fall-2021/blob/main/solutions-by-participants/challenge-4/4C_LukasBotsch.ipynb. (2021).

[4] Daniel Burgarth, Paolo Facchi, Hiromichi Nakazato, Saverio Pascazio, and Kazuya Yuasa. 2020. Quantum zeno dynamics from general quantum operations. *Quantum* 4 (2020), 289.

[5] Andrey Bychkov, Opal Issan, Gleb Pogudin, and Boris Kramer. 2024. Exact and Optimal Quadratization of Nonlinear Finite-Dimensional Nonautonomous Dynamical Systems. *SIAM Journal on Applied Dynamical Systems* 23, 1 (2024), 982–1016.

[6] Steven A Cuccaro, Thomas G Draper, Samuel A Kutin, and David Petrie Moulton. 2004. A new quantum ripple-carry addition circuit. *arXiv preprint quant-ph/0410184* (2004).

[7] Nike Dattani. 2019. Quadratization in discrete optimization and quantum mechanics. *arXiv preprint arXiv:1901.04405* (2019).

[8] Pierre Dupuy de la Grand'rive and Jean-Francois Hullo. 2019. Knapsack problem variants of qaoa for battery revenue optimisation. *arXiv preprint arXiv:1908.02210* (2019).

[9] Thomas G Draper. 2000. Addition on a quantum computer. *arXiv preprint quant-ph/0008033* (2000).

[10] Paolo Facchi and Saverino Pascazio. 2008. Quantum Zeno dynamics: mathematical and physical aspects. *Journal of Physics A: Mathematical and Theoretical* 41, 49 (2008), 493001.

[11] Edward Farhi, Jeffrey Goldstone, and Sam Gutmann. 2014. A quantum approximate optimization algorithm. *arXiv preprint arXiv:1411.4028* (2014).

[12] Ambros Gleixner, Gregor Hendel, Gerald Gamrath, Tobias Achterberg, Michael Bastubbe, Timo Berthold, Philipp M. Christophel, Kati Jarck, Thorsten Koch, Jeff Linderoth, Marco Lübbecke, Hans D. Mittelmann, Derya Ozyurt, Ted K. Ralphs, Domenico Salvagnin, and Yuji Shinano. 2021. MIPLIB 2017: Data-Driven Compilation of the 6th Mixed-Integer Programming Library. *Mathematical Programming Computation* (2021). https://doi.org/10.1007/s12532-020-00194-3

[13] Dylan Herman, Ruslan Shaydulin, Yue Sun, Shouvanik Chakrabarti, Shaohan Hu, Pierre Minssen, Arthur Rattew, Romina Yalovetzky, and Marco Pistoia. 2022. Portfolio Optimization via Quantum Zeno Dynamics on a Quantum Processor. *arXiv preprint arXiv:2209.15024* (2022).

[14] Dylan Herman, Ruslan Shaydulin, Yue Sun, Shouvanik Chakrabarti, Shaohan Hu, Pierre Minssen, Arthur Rattew, Romina Yalovetzky, and Marco Pistoia. 2023. Constrained optimization via quantum Zeno dynamics. *Communications Physics* 6, 1 (Aug. 2023). https://doi.org/10.1038/s42005-023-01331-9

[15] Ryan L'Abbate, Anthony D'Onofrio, Samuel Stein, Samuel Yen-Chi Chen, Ang Li, Pin-Yu Chen, Juntao Chen, and Ying Mao. 2024. A quantum-classical collaborative training architecture based on quantum state fidelity. *IEEE Transactions on Quantum Engineering* (2024).

[16] Zhiding Liang, Gang Liu, Zheyuan Liu, Jinglei Cheng, Tianyi Hao, Kecheng Liu, Hang Ren, Zhixin Song, Ji Liu, Fanny Ye, et al. 2024. Graph Learning for Parameter Prediction of Quantum Approximate Optimization Algorithm. *arXiv preprint arXiv:2403.03310* (2024).

[17] Zhiding Liang, Zhixin Song, Jinglei Cheng, Zichang He, Ji Liu, Hanrui Wang, Ruiyang Qin, Yiru Wang, Song Han, Xuehai Qian, et al. 2023. Hybrid gate-pulse model for variational quantum algorithms. In *2023 60th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 1–6.

[18] Satoshi Morita and Hidetoshi Nishimori. 2008. Mathematical foundation of quantum annealing. *J. Math. Phys.* 49, 12 (2008).

[19] Lidia Ruiz-Perez and Juan Carlos Garcia-Escartin. 2017. Quantum arithmetic with the quantum Fourier transform. *Quantum Information Processing* 16 (2017), 1–14.

[20] Siwei Tan, Congliang Lang, Liang Xiang, Shudi Wang, Xinghui Jia, Ziqi Tan, Tingting Li, Jieming Yin, Yongheng Shang, Andre Python, et al. 2023. QuCT: A Framework for Analyzing Quantum Circuit by Extracting Contextual and Topological Features. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*. 494–508.

[21] Vlatko Vedral, Adriano Barenco, and Artur Ekert. 1996. Quantum networks for elementary arithmetic operations. *Physical Review A* 54, 1 (1996), 147.

[22] Ke Wan and Yiwen Liu. 2023. Hybrid Quantum Algorithms integrating QAOA, Penalty Dephasing and Zeno Effect for Solving Binary Optimization Problems with Multiple Constraints. *arXiv preprint arXiv:2305.08056* (2023).

[23] Hanrui Wang, Pengyu Liu, Jinglei Cheng, Zhiding Liang, Jiaqi Gu, Zirui Li, Yongshan Ding, Weiwen Jiang, Yiyu Shi, Xuehai Qian, et al. 2022. Quest: Graph transformer for quantum circuit reliability estimation. *arXiv preprint arXiv:2210.16724* (2022).

[24] Hanrui Wang, Pengyu Liu, Daniel Bochen Tan, Yilian Liu, Jiaqi Gu, David Z Pan, Jason Cong, Umut A Acar, and Song Han. 2024. Atomique: A quantum compiler for reconfigurable neutral atom arrays. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 293–309.

[25] Hongye Yu and Tzu-Chieh Wei. 2021. Quantum Zeno approach for molecular energies with maximum commuting initial Hamiltonians. *Physical Review Research* 3, 1 (2021), 013104.

[26] Jia Zhao. 2021. A revisit of the energy quadratization method with a relaxation technique. *Applied Mathematics Letters* 120 (2021), 107331.

[27] Jun Zhuang, Jack Cunningham, and Chaowen Guan. 2024. Improving Trainability of Variational Quantum Circuits via Regularization Strategies. *arXiv preprint arXiv:2405.01606* (2024).